Procedural Programming

Part 1: Focus on Functions

The idea that programs are made up of interconnected functions to manage and process data.

Procedural Programming

Procedural programming organizes a program around procedures or functions that perform specific tasks.

Basic **structured programming** focuses on clear control flow using sequence, selection, and repetition, while procedural programming adds stronger organization by dividing that logic into reusable functions.

The goal is to break a larger program into smaller procedures that can be understood and tested separately.

Focus on Functions

Procedural programming relies heavily on functions to organize a program into smaller sections of behaviour.

Each function usually receives data, performs a task, and may return a result.

Procedural programming simply uses functions as its main way of structuring program logic.

Using Functions for Input/Output Machines

A function can be thought of as a small input/output machine.

It receives values through parameters, processes those values, and can return a result.

This makes the function easier to reuse because the same logic can work with different inputs.

```
def calculate_area(width, height):
    area = width * height
    return area


result = calculate_area(5, 3)
print(result)
```

OUTPUT

```
15
```

Atomization

Atomization means breaking a larger program into smaller functions that each handle one part of the problem.

Instead of keeping all of the program's logic together, each major task becomes its own procedure.

This makes the program easier to read, test, and change.

Atomization

```
# Without Functions

name = input("Enter your name: ")
score1 = int(input("First score: "))
score2 = int(input("Second score: "))

average = (score1 + score2) / 2

print(f"{name}'s average is {average}.")
```

```
# With Functions

def get_name():
    return input("Enter your name: ")

def get_score():
    return int(input("Enter a score: "))

def calculate_average(score1, score2):
    return (score1 + score2) / 2

def display_result(name, average):
    print(f"{name}'s average is {average}.")

name = get_name()
score1 = get_score()
score2 = get_score()

average = calculate_average(score1, score2)

display_result(name, average)
```

One Function, One Task

A useful procedural programming guideline is that each function should have one clear responsibility.

Functions can contain several lines of code, but those lines should all contribute to the same task.

Separating unrelated jobs makes functions easier to understand, reuse, and test.

One Function, One Task

```python
# One Function, One Task

def get_price():
    return float(input("Enter the item price: "))

def calculate_tax(price):
    return price * 0.05

def calculate_total(price, tax):
    return price + tax

def display_total(total):
    print(f"Total: ${total:.2f}")

price = get_price()
tax = calculate_tax(price)
total = calculate_total(price, tax)
display_total(total)
```

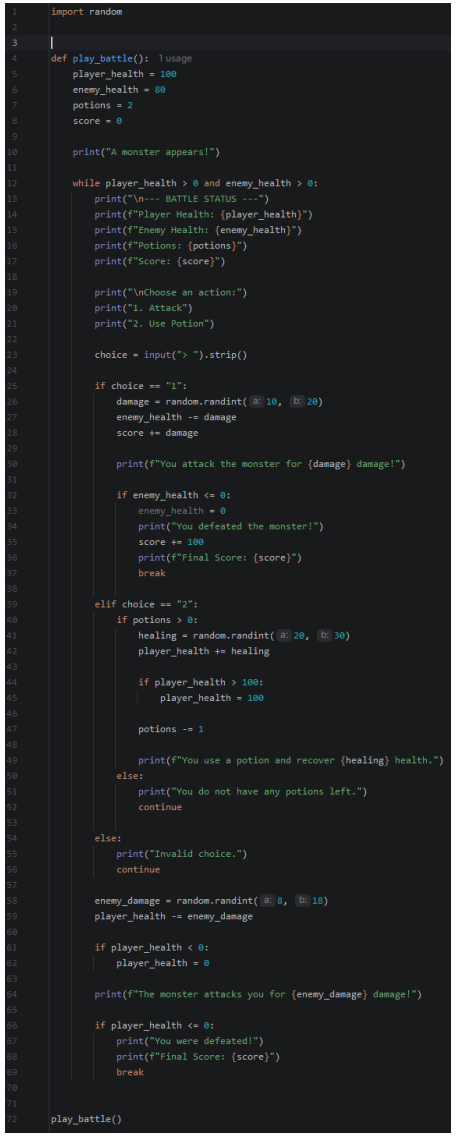
Note: This example is an exaggeration. Keeping text to fit on a slide limits a function already to be kept pretty simple

```python
# One Function Does Everything

def process_purchase():
    price = float(input("Item price: "))
    tax = price * 0.05
    total = price + tax
    print(f"Total: ${total:.2f}")


process_purchase()
```

One Bloated Mega Function



```python
import random

def play_battle():  1 usage
    player_health = 100
    enemy_health = 80
    potions = 2
    score = 0

    print("A monster appears!")

    while player_health > 0 and enemy_health > 0:
        print("\n--- BATTLE STATUS ---")
        print(f"Player Health: {player_health}")
        print(f"Enemy Health: {enemy_health}")
        print(f"Potions: {potions}")
        print(f"Score: {score}")

        print("\nChoose an action:")
        print("1. Attack")
        print("2. Use Potion")

        choice = input("> ").strip()

        if choice == "1":
            damage = random.randint( a: 10,  b: 20)
            enemy_health -= damage
            score += damage

            print(f"You attack the monster for {damage} damage!")

            if enemy_health <= 0:
                enemy_health = 0
                print("You defeated the monster!")
                score += 100
                print(f"Final Score: {score}")
                break

        elif choice == "2":
            if potions > 0:
                healing = random.randint( a: 20,  b: 30)
                player_health += healing

                if player_health > 100:
                    player_health = 100

                potions -= 1

                print(f"You use a potion and recover {healing} health.")
            else:
                print("You do not have any potions left.")
                continue

        else:
            print("Invalid choice.")
            continue

        enemy_damage = random.randint( a: 8,  b: 18)
        player_health -= enemy_damage

        if player_health < 0:
            player_health = 0

        print(f"The monster attacks you for {enemy_damage} damage!")

        if player_health <= 0:
            print("You were defeated!")
            print(f"Final Score: {score}")
            break


play_battle()
```

Same Program Broken Into Smaller Functions



```python
def display_status(player_health, enemy_health, potions, score):
    print("\n--- BATTLE STATUS ---")
    print(f"Player Health: {player_health}")
    print(f"Enemy Health: {enemy_health}")
    print(f"Potions: {potions}")
    print(f"Score: {score}")
```

```python
def use_potion(player_health: {__iadd__}, potions: {__le__, __isub__}):
    if potions <= 0:
        print("You do not have any potions left.")
        return player_health, potions, False

    healing = random.randint( a: 20,  b: 30)
    player_health += healing

    if player_health > 100:
        player_health = 100

    potions -= 1

    print(f"You use a potion and recover {healing} health.")

    return player_health, potions, True
```

```python
def enemy_attack(player_health: {__isub__}):  1 usage  new *
    damage = random.randint( a: 8,  b: 18)
    player_health -= damage

    if player_health < 0:
        player_health = 0

    print(f"The monster attacks you for {damage} damage!")

    return player_health
```

```python
def get_player_action():  1 usage  n
    print("\nChoose an action:")
    print("1. Attack")
    print("2. Use Potion")

    return input("> ").strip()
```

```python
def display_game_over(player_health: {__gt__}, score):
    if player_health > 0:
        print("You defeated the monster!")
        score += 100
    else:
        print("You were defeated!")

    print(f"Final Score: {score}")

    return score
```

```python
def player_attack(enemy_health: {__isub__}, score: {__iadd__}):
    damage = random.randint( a: 10,  b: 20)
    enemy_health -= damage
    score += damage

    if enemy_health < 0:
        enemy_health = 0

    print(f"You attack the monster for {damage} damage!")

    return enemy_health, score
```

```python
def play_battle():  1 usage  new *
    player_health = 100
    enemy_health = 80
    potions = 2
    score = 0

    print("A monster appears!")

    while player_health > 0 and enemy_health > 0:
        display_status(player_health, enemy_health, potions, score)

        choice = get_player_action()

        if choice == "1":
            enemy_health, score = player_attack(enemy_health, score)

        elif choice == "2":
            player_health, potions, action_completed = use_potion(
                player_health,
                potions
            )

            if not action_completed:
                continue

        else:
            print("Invalid choice.")
            continue

        if enemy_health <= 0:
            break

        player_health = enemy_attack(player_health)

    display_game_over(player_health, score)
```

Stub Programming

Stub programming means creating the functions a program will need before writing their full behaviour.

A stub usually contains **pass** or a temporary placeholder so the overall structure of the program can be planned first.

This allows programmers to design how functions connect before completing each part.

```
def get_player_name():
    pass

def roll_dice():
    pass

def move_player():
    pass

def display_board():
    pass

def main():
    get_player_name()
    roll_dice()
    move_player()
    display_board()

main()
```